

Ministère de l'Enseignement Supérieur et de la Recherche
CY Cergy Paris Université UFR Sciences et Techniques
Département Informatique



Rapport de Projet : POO & Java

Thème

Gestionnaire de métadonnées de fichiers MP3 et Playlists.

Réalisé par :

- ZIRMI Amira
- MOKRANE Maria

Supervisé par :

- LEMAIRE Marc
- TANXIAO Liu
- BOURDON Jean-luc

Table des matières

1.Introduction.....	3
1.1.Contexte et problématique du projet.....	3
1.2.Présentation générale du projet.....	3
1.3.Objectifs du projet.....	3
2.Gestion de projet.....	3
2.1.Répartition des tâches.....	3
3.Analyse et conception.....	4
3.1.Analyse.....	4
3.2.Conception.....	5
4.Réalisation technique (approche par solution).....	6
4.1.Validation d'un fichier MP3 avant analyse.....	6
4.2.Extraction des métadonnées des fichiers MP3.....	7
4.3.Recherche récursive des fichiers MP3 dans un dossier.....	7
4.4.Création et gestion des playlists.....	7
4.5.Bibliothèques externes utilisées.....	8
4.6.Logiciels utilisés.....	8
5.Manuel d'utilisation.....	8
5.1.Mode CLI.....	8
5.2.Mode GUI.....	9
6.Bilan.....	10
6.1.État d'avancement.....	10
6.2.Difficultés rencontrées et solutions.....	10
6.3.Compétences acquises.....	10
7.Conclusion.....	10
7.1. Résumé général de la solution.....	10
7.2. Perspectives.....	10

1.Introduction

1.1.Contexte et problématique du projet

À l'heure où les collections musicales numériques se multiplient rapidement, organiser efficacement ses fichiers audio est devenu un véritable défi. Parmi les formats les plus répandus, le MP3 se distingue par sa capacité à compresser le flux audio tout en conservant une qualité sonore acceptable, ce qui en fait un format pratique pour stocker et écouter de la musique. Au-delà du flux audio lui-même, les fichiers MP3 contiennent des **métadonnées** stockées dans des **tags ID3**, telles que le titre, l'artiste, l'album et le genre. Ces informations sont essentielles pour identifier, organiser et exploiter efficacement une collection musicale, facilitant ainsi la recherche et la gestion des fichiers audio.

La problématique principale de ce projet est donc la suivante :

Comment concevoir une application Java capable d'analyser des fichiers MP3, d'en extraire les métadonnées et de générer automatiquement des playlists dans différents formats, tout en étant simple d'utilisation et robuste face aux erreurs ?

1.2.Présentation générale du projet

Ce projet consiste à développer une application Java permettant :

- D'analyser des fichiers MP3.
- D'afficher leurs métadonnées.
- De parcourir récursivement des dossiers musicaux.
- De créer automatiquement des playlists dans plusieurs formats standards (XSPF, M3U8 et JSPF).

L'application propose deux modes d'utilisation : en **mode console (CLI)** et en **mode graphique (GUI)**.

1.3.Objectifs du projet

Les objectifs principaux sont :

- Appliquer les concepts de la POO (classes, héritage, encapsulation, polymorphisme).
- Concevoir une architecture logicielle claire.
- Manipuler des fichiers, des dossiers et des formats standards en Java.
- Utiliser des bibliothèques externes.
- Concevoir une architecture modulaire et maintenable.

2.Gestion de projet

2.1.Répartition des tâches

Le projet a été réalisé en binôme avec une répartition des tâches permettant un travail parallèle et complémentaire.

Le projet s'est déroulé selon les étapes suivantes :

1. Compréhension du sujet et analyse des besoins.
2. Conception UML.
3. Développement du noyau applicatif.
4. Implémentation du mode CLI et du mode GUI.
5. Tests et corrections.

6. Rédaction du rapport et préparation de la démonstration.

Un diagramme de Gantt a été réalisé afin de planifier ces différentes phases et de suivre l'avancement du projet.

Diagramme de Gantt



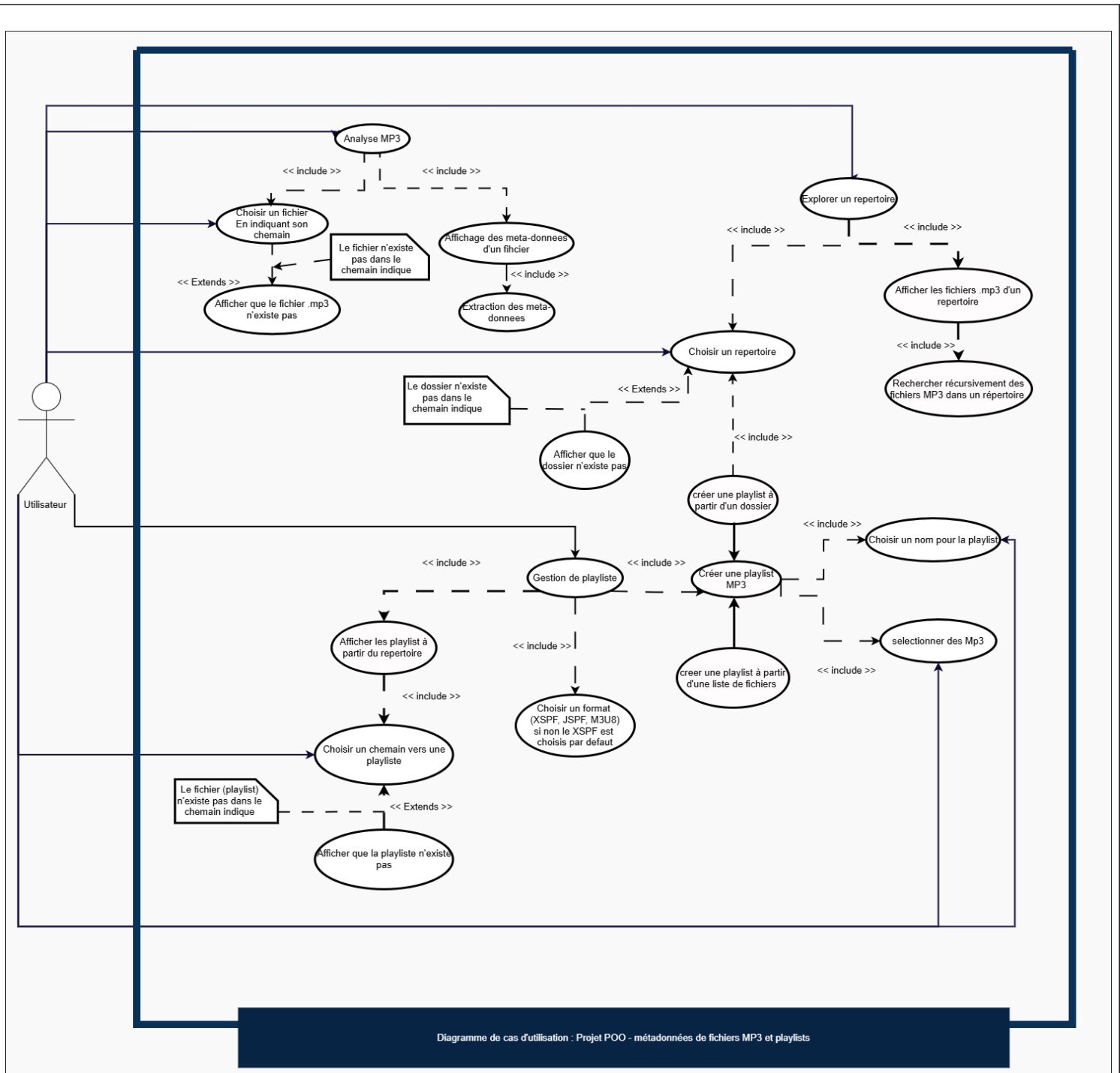
3. Analyse et conception

3.1. Analyse

L'analyse des besoins a pour objectif d'identifier précisément ce que le système doit faire et sous quelles contraintes, indépendamment des choix techniques.

- Le système doit permettre à l'utilisateur de choisir un fichier MP3 et d'en analyser le contenu afin d'extraire : les métadonnées (titre, artiste et album...) et certaines informations techniques (durée du morceau).
- Le système doit permettre à l'utilisateur de fournir un dossier contenant des fichiers musicaux afin de parcourir récursivement le dossier et ses sous-dossiers, identifier uniquement les fichiers MP3 valides et afficher la liste des fichiers détectés.
- À partir d'un ensemble de fichiers MP3 personnalisées ou bien à partir d'un dossier contenant des fichiers mp3, le système doit permettre de : générer une playlist, choisir le format de sortie et enregistrer la playlist et d'ajouter une musique à une playlist existante (pour le mode GUI).

Le diagramme des cas d'utilisation résumant ces fonctionnalités est présenté ci-dessous :

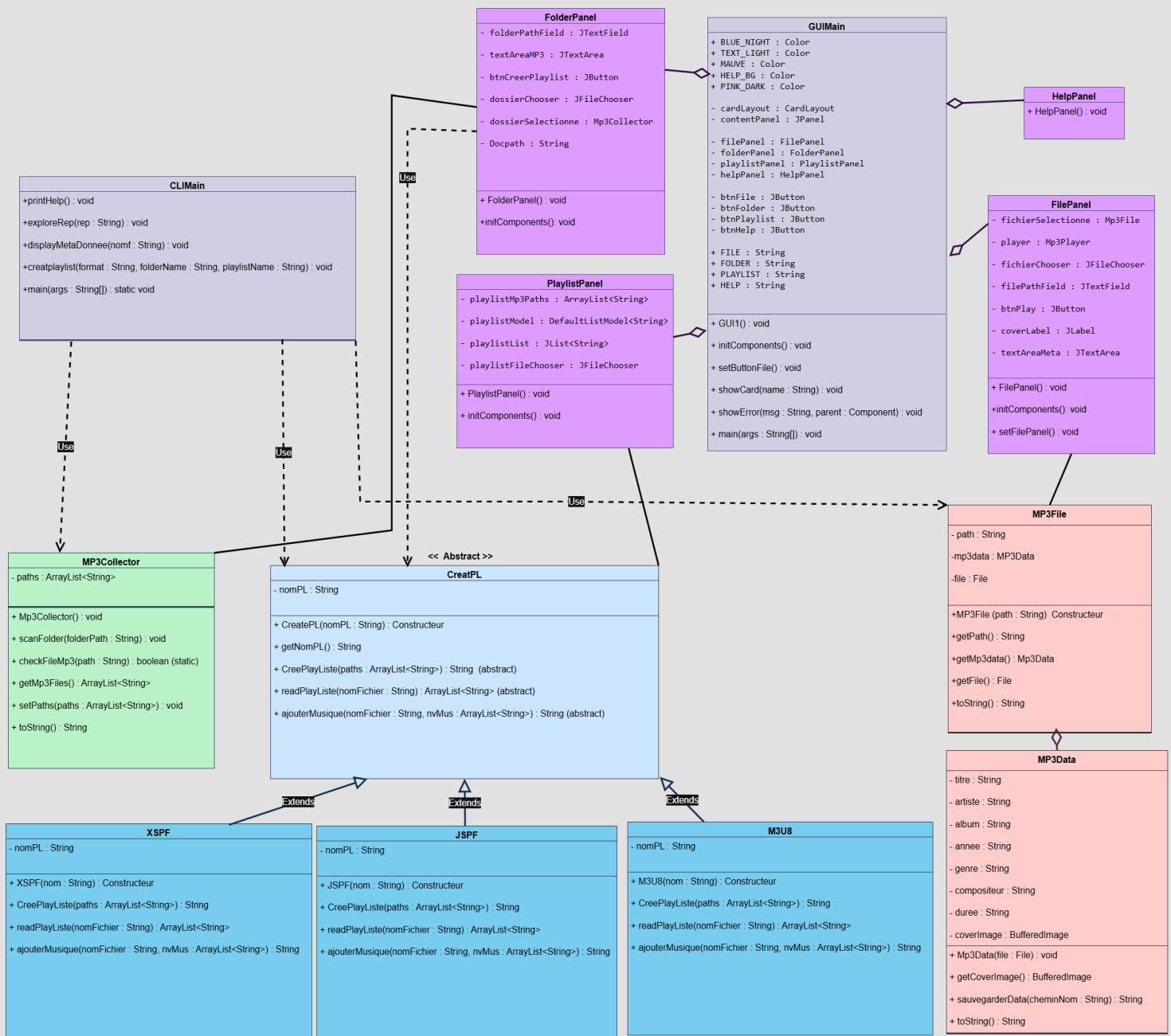


3.2.Conception

La conception repose sur une architecture orientée objet structurée autour de plusieurs entités principales :

- une classe gérant les métadonnées et une autre représentant un fichier MP3 ;
- une classe pour la recherche récursive des mp3 dans un dossier ;
- des classes dédiées aux formats de playlists ;
- des classes pour les interfaces CLI et GUI.

Le diagramme de classe résumant ces fonctionnalités est présenté ci-dessous :



4. Réalisation technique (approche par solution)

Cette partie décrit les **mécanismes principaux mis en œuvre** pour répondre aux besoins du projet.

4.1. Validation d'un fichier MP3 avant analyse

Avant toute extraction de métadonnées, il est indispensable de vérifier que le fichier fourni est bien un MP3 valide. L'application effectue donc une vérification préalable : le fichier doit exister, être un fichier régulier et posséder l'extension .mp3.

Afin de s'assurer de son authenticité, le type réel du fichier est ensuite identifié à l'aide de la bibliothèque **Apache Tika**, qui analyse la signature interne du fichier plutôt que son nom. Cette analyse permet de déterminer le type MIME réel, qui doit correspondre à audio/mpeg.

4.2.Extraction des métadonnées des fichiers MP3

Pour l'extraction des métadonnées, la solution repose sur une **séparation claire des responsabilités**.

Deux types d'objets interviennent :

- un objet représentant le **fichier MP3** ;
- un objet représentant les **métadonnées** du fichier.

Lorsqu'un fichier MP3 est analysé, l'application :

1. vérifie d'abord que le fichier est valide ;
2. lit ensuite ses métadonnées.

L'extraction des métadonnées est réalisée à l'aide de la bibliothèque **jAudioTagger**, qui permet d'accéder directement aux tags ID3 des fichiers MP3. Les informations descriptives sont obtenues via l'objet Tag, tandis que les données techniques, comme la durée, sont récupérées grâce à l'AudioHeader.

Cette méthode offre une solution standardisée et fiable, évitant l'analyse manuelle de la structure binaire des fichiers.

4.3.Recherche récursive des fichiers MP3 dans un dossier

Pour permettre l'exploration de bibliothèques musicales complètes, l'application implémente une **recherche récursive** dans les dossiers.

Le principe est le suivant :

- si un élément est un dossier, il est parcouru récursivement ;
- si un élément est un fichier, il est testé pour vérifier s'il s'agit d'un MP3 valide ;
- seuls les fichiers validés sont conservés.

Cette approche permet d'explorer efficacement des arborescences complexes contenant de nombreux sous-dossiers.

4.4.Création et gestion des playlists

L'application définit une structure commune pour les playlists, puis implémente une classe spécifique pour chaque format :

Nom du format	XSPF	JSPF	M3U8
Exemple du format	<pre><?xml version="1.0" encoding="UTF-8"?> <playlist version="1" xmlns="http://xspf.org/ns/0/"> <trackList> <track> <location>file:///path/fichier.mp3</location> </track> </trackList> </playlist></pre>	<pre>{ "playlist": { "title": "Nom de la playlist", "track": [{ "location": "file:///path/fichier.mp3" }] } }</pre>	<pre>#EXTM3U file:///path/fichier.mp3</pre>

Quel que soit le format choisi, le principe reste le même : on a récupération de la liste des fichiers MP3, génération du contenu de la playlist selon la syntaxe du format, sauvegarde dans un fichier.

Cette organisation facilite la maintenance du code et permet d'ajouter facilement de nouveaux formats.

4.5. Bibliothèques externes utilisées

Pour ce projet, plusieurs bibliothèques Java externes ont été intégrées afin de simplifier le développement et d'améliorer la fiabilité du code :

1. **Jaudiotagger** : utilisée pour accéder aux métadonnées (ID3 tags) des fichiers MP3, telles que le titre, l'artiste... Elle offre une interface standardisée et fiable pour l'extraction des informations audio, éviter de gérer manuellement la structure binaire des fichiers audio.
2. **Apache Tika** : sert à vérifier le type réel des fichiers avant traitement, en analysant leur signature interne pour déterminer le type MIME. Cette validation permet de garantir que seuls des fichiers MP3 authentiques (audio/mpeg) sont analysés, renforçant la sécurité et la robustesse de l'application.
3. **JLayer** : utilisée pour la lecture de fichiers MP3. Légère et compatible avec Java SE 21, elle permet de jouer les fichiers audio facilement sans nécessiter la gestion directe des flux ou des codecs, offrant une solution simple et efficace pour les fonctionnalités de lecture dans l'interface graphique.

Ces bibliothèques ont été choisies pour leur stabilité, leur compatibilité avec Java SE 21, et leur capacité à réduire la complexité du code tout en fournissant des fonctionnalités fiables pour la manipulation et la lecture des fichiers MP3.

4.6. Logiciels utilisés

Pour le développement, la conception et la gestion de ce projet, plusieurs logiciels et outils ont été utilisés :

1. **Lucidchart** : utilisé pour la réalisation du diagramme de GANTT.
2. **Draw.io** : utilisé pour créer les diagrammes de cas d'utilisation et les diagrammes de classe.
3. **IntelliJ IDEA** : environnement de développement intégré (IDE) utilisé pour coder l'application en Java.
4. **VLC Media Player** : pour ouvrir et tester les playlists générées afin de vérifier leur bon fonctionnement.
5. **Valideur XSPF en ligne** : pour valider les fichiers (playlist) au format XSPF.
6. **LibreOffice** : utilisé pour rédiger le rapport du projet.
7. **OBS Studio** : logiciel de capture vidéo utilisé pour enregistrer la démonstration du projet.
8. **Logiciels de communication et de gestion de projet** : e-mail, Discord, Google Drive et Click UP.

5. Manuel d'utilisation

5.1. Mode CLI

Option	Description
-h ou --help	Affiche l'aide et les options possibles
-f <fichier>	Spécifie un fichier MP3 à analyser
-d <dossier>	Spécifie un dossier à explorer pour générer une playlist
-o <fichier>	Indique un fichier de sortie pour sauvegarder la playlist générée
--format	Permet de spécifier le format de la playlist à générer. Les formats acceptés sont : XSPF , JPSF , M3U8 . Cette option doit être utilisée

avant d'indiquer le fichier de sortie avec -o.
Si aucun format n'est spécifié, le programme utilise par défaut le format **XSPF**.

Exemple d'exécution :

```
PS C:\Users\DELL\Desktop> java -jar ./cli.jar
Erreur : paramètres manquants.
Tapez -h ou --help pour afficher l'aide.
PS C:\Users\DELL\Desktop> java -jar ./cli.jar -h
```

Modes d'utilisation (console) :

```
-h | --help
Affiche l'aide.

-d <dossier>
Parcourt récursivement le dossier spécifié et affiche
tous les fichiers MP3 valides trouvés.

-f <fichier.mp3>
Affiche les métadonnées du fichier MP3 donné.

-d <dossier> -o <fichier_cible.format>
Génère une playlist à partir du dossier spécifié,
en utilisant le format par défaut XSPF,
et l'enregistre dans le fichier cible.

-d <dossier> -format <xspf|m3u|txt> -o <fichier_cible.format>
Génère une playlist à partir du dossier spécifié,
en utilisant le format choisi (XSPF, JSPF ou M3U8),
et l'enregistre dans le fichier cible.
```

Mode graphique :
java -jar gui.jar
Lance l'interface graphique.

```
PS C:\Users\DELL\Desktop> java -jar ./cli.jar -f ./music/L-0-V-E.mp3
```

```
fichier      : ./music/L-0-V-E.mp3
titre       : L-0-V-E
artiste     : Nat King Cole
album      : L-0-V-E
année      : 19/05/2016
genre      :
compositeur :
durée      : 2:34
```

```
PS C:\Users\DELL\Desktop> java -jar ./cli.jar -d ./music
Dynasty (Official Music Video) MIIA.mp3
L-0-V-E.mp3
Taylor Swift - The Fate of Ophelia (Official Music Video).mp3
```

```
PS C:\Users\DELL\Desktop> java -jar ./cli.jar -d ./music --xspf -o myplaylist.xspf
```

Votre playliste est cree avec succee

5.2.Mode GUI

Organisation de l'interface (CardLayout)

L'interface est composée de **quatre panels** :

1. **Fichier MP3** : choisir un fichier MP3, afficher ses métadonnées et les sauvegarder dans un fichier.
2. **Dossier MP3** : sélectionner un dossier, afficher tous les MP3, et générer une playlist à partir des fichiers trouvés.
3. **Playlist** : créer une nouvelle playlist, ajouter ou supprimer des MP3, ou ouvrir une playlist existante.
4. **Help** : afficher le fonctionnement de l'application et les instructions pour l'utilisateur.

Exemple :



6.Bilan

6.1.État d'avancement

L'application est fonctionnelle et répond à l'ensemble des exigences du cahier des charges:

- Extraction des métadonnées ID3 via jAudioTagger.
- Exploration récursive des dossiers avec filtrage par extension et type MIME via Apache Tika.
- Génération de playlists aux formats XSPF, JSPF et M3U8.
- Modes CLI et GUI complets avec gestion des options (-h, -f, -d, -o).
- Bonus réalisé : Lecture audio intégrée dans la GUI via JLayer et affichage des couvertures d'album.

6.2.Difficultés rencontrées et solutions

- Limitation des tests pour JSPF : Alors que les playlists XSPF et M3U8 peuvent être facilement testées et lues avec des logiciels comme VLC Media Player, le format JSPF n'est pas supporté par la majorité des lecteurs audio standards.
- Fichiers MP3 corrompus : Certains fichiers avaient l'extension .mp3 mais étaient illisibles. Solution : Utilisation d'Apache Tika pour vérifier le type MIME audio/mpeg avant traitement.

6.3.Compétences acquises

- Conception logicielle : Modélisation de systèmes complexes via UML (cas d'utilisation et diagrammes de classes).
- Gestion de projet : Planification rigoureuse via un diagramme de GANTT et répartition équitable des tâches
- Maîtrise technique : Intégration de bibliothèques externes dans un projet Java SE 21 et gestion des flux d'entrée/sortie (I/O).
- Ce projet a été formateur par sa dimension concrète. Il nous a obligées à sortir du cadre purement théorique pour nous confronter aux standards du monde réel.

7.Conclusion

7.1. Résumé général de la solution

Nous avons conçu une application modulaire et robuste capable de transformer une simple arborescence de fichiers en une bibliothèque musicale organisée. En séparant le cœur métier (extraction et recherche) des interfaces (CLI et GUI), notre solution offre une grande flexibilité d'utilisation tout en garantissant l'intégrité des métadonnées extraites.

7.2. Perspectives

Bien que le cahier des charges soit rempli, plusieurs axes d'amélioration sont envisageables :

- Connectivité : Ajouter une fonctionnalité permettant de récupérer les pochettes d'albums manquantes via une API de métadonnées en ligne.
- Fonctionnalités : Étendre le support à d'autres formats audio comme le FLAC ou le WAV.